



۸۱۰۱۰۱۴۹۲

شبکه‌های کامپیوتری
نام و نام خانوادگی: علیرضا کریمی



نوت ۱۶ آذر

فصل پنجم: لایه *Network* بخش *Control – Plane* اسلاید ۱۶ تا ۳۹ [۱]

خب ابتدا مثالی از الگوریتم دایکسترا حل شد. برای هر نود، وقتی که عمل آپدیت را انجام می‌دهیم، نود قبلی که با آن به اون نود رسیده را هم در جدول مینویسیم، که اینگونه بتوانیم کوتاه ترین مسیر از مقصد تا مبدا را پیدا کنیم چون هر نود، نود قبلی خود را در مسیر کمینه میداند. مثلا اگر در ستون $D(z)$ در ردیف آخرش نوشته باشیم y , 12, یعنی $y = p(z)$ و y نود قبل از z در مسیر کمینه است و فاصله تا z هم در مسیر کمینه ۱۲ است. اردر الگوریتم دایکسترا، چون در هر *iteration* باید تمام نود های دیده نشده را به نوعی بررسی کنیم، و این کار را در n تا *iteration* انجام دهیم، از اردر $(n-1)/2 \times n$ یا همان $O(n^2)$ است.

مشکل *oscillations* در الگوریتم دایکسترا

چیزی که فهمیدم، احتمالا اینه که اگر مسیر های مختلف تقریبا یک بار ترافیکی داشته باشند، و می خواهیم بدانیم ترافیکمان را به کدام مسیر بفرستیم، هر بار که یک مسیری را انتخاب کنیم، آن مسیر دیگر، بار کمتری خواهد داشت و به آن سوییچ میکنیم و این کار به طور مداوم و پی در پی انجام میشود و هی مسیر را عوض میکنیم.

الگوریتم های روتینگ *distance vector*

خصوصیات:

- *distributed* هستند و هر گره فقط با همسایه هایش صحبت میکند.
 - *iterative* هستند و اجرای الگوریتم به طور تکراری ادامه میابد تا وقتی که دیگر اطلاعاتی جا به جا نشوند. نیاز به سیگنال برای توقف آن نیست، خودشان خودکار خاتمه پیدا میکنند.
 - *Asynchronous* است و هر گره اطلاعات را در هر لحظه که بخواهد به همسایه هایش میدهد. و نیاز به هماهنگی قبلی برای اجرای الگوریتم نیست.
- این الگوریتم ها طبق روش *bellman – ford* که یک نوع *dynamic programming* است، عمل می کنند.

خب چند تا تعریف داریم. مثلاً

$$d_x = (y) := \text{cost of least} - \text{cost path from } x \text{ to } y$$

که کمترین هزینه از نود x به y است.
و داریم:

$$D_x(y) = \text{estimate of least cost from } x \text{ to } y$$

که درواقع تخمین محلی از دیدگاه نود x است که فاصله اش تا y چقدر است. در هر مرحله با اطلاعاتی که از همسایه ها میگیرد، مثلاً بردار فاصله همسایه ها را میگیرد و طبق آن این $D_x = D_x(y) | y \in N$ را آپدیت میکند. درواقع از آنجایی که حداقل cost لینک های محلی (مستقیم متصل به خودش) را میداند، که برابر $c(x, v)$ است، اینگونه آپدیت را در هر مرحله انجام میدهد:

$$D_x(y) = \min\{c(x, v) + D_v(y)\} \text{ for every } v \text{ as a neighbor}$$

یعنی تخمین همسایه ها را به علاوه لینک محلی خودش میکند و اگر تغییر کرده بود آن را آپدیت میکند مثلاً. پس هر نود همینطوری بردار فاصله خودش را به بقیه میدهد و در هر مرحله آنها طبق اطلاعات همسایه ها آپدیت میکنند تا در نهایت D_x به d_x و مقدار اصلی converge کند و دیگر چیزی تغییر نکند.

هر iteration آسنکرون local برای آپدیت کردن بردار فاصله یک نود، به دو دلیل میتواند رخ دهد.

۱. هزینه لینک محلی آن نود تغییر کند.

۲. همسایه ها اطلاعات تغییر یافته بردار فاصله خودشان را برای نود بفرستند.

در این دو حالت، دوباره محاسبه بردار فاصله را انجام میدهد و در صورت تغییر، همسایه هایش را notify میکند. حال یک تعریف دییگر هم داریم به اسم جدول فاصله distance table که به صورت مثلاً این شکلی تعریف می‌شود:
 $d_E(A, B)$ یعنی اینکه فاصله E از طریق نود بعدی B به A چقدر است. محاسبه آن میتواند به این شکل باشد:

$$d_E(A, B) = c(E, B) + \min\{d_B(A, w)\} \text{ for all } w$$

حواست باشه جدول فاصله با بردار فاصله فرق داره ولی خب به کمک همون اطلاعات بردار فاصله میتوان آن را تا حدی آپدیت کرد.

مثلاً جدول $T_E()$ یعنی برای نود E می‌خوایم بررسی کنیم. ردیف ها در جدول فاصله، مقصد ما هستند و ستون ها، همان نود بعدی که می‌خوایم از طریق آن به آن مقصد ها بریم.

پس کمترین عدد در هر ردیف، میتواند کمترین مسیر از مسیر آن نود به مقصد را برای ما مشخص کند. البته ممکن است با این آپدیت کردن های جدول فاصله به عددهایی بخوریم که حاصل *loop back* هستند. مثلا E میخواد از طریق نود B به A بره. توی خود مسیر B به A هم از E بگذریم. یعنی دو بار توی مسیرمون E بوده و بعش برگشتیم وسط مسیر. پس عددمون درست نیست. حالا ممکن است *cost* لینک های همسایه ها تغییر کند.

- اگر که هزینه یک لینک همسایه کم شود، *good news* است و سریع پخش میشود و جدول فاصله بقیه را آپدیت میکند.

- اگر هزینه یک لینک زیاد شود، *bad news* است و ممکن است به تعداد دور های زیادی، جدول های فاصله به صورت *iterative* و یکی در میون، آپدیت بشن و دونه دونه افزایش پیدا میکنند این مقادیر جدول فاصله، تا زمانی که کلا *cost* کمینه در اون ردیف از جدول عوض شود و یکی دیگه شود و اینگونه ثابت بشود و بماند. این مشکل به *count to infinity* هم معروف است. پس ممکن است تا بینهایت پیش رود.

مثال صفحه ۳۴ رو حتما ببین. یک لینک کلا قطع شده و بقیه میخوان خودشون رو آپدیت کنن با مقادیر قبلی و غلط قبلی بقیه همسایه ها که هنوز نمیدونن اون لینک قطع شده. البته اگه دقت کنی، میبینی که اینجا هم مشکل *loop back* به نوعی تاثیر گذار است.

دلیل این مشکل همگرایی خبر بد دو مورد است:

۱. پیام ها به صورت ناهمگام و آسنکرون منتقل میشوند
۲. اینکه هر نود، بردار فاصله را به همسایه هایش میدهد و نه کل جدول فاصله اش را

راه حل *Poison Reverse*

اینکه Z از طریق Y به X کمترین مسیر داشته باشد، Z به Y میگوید که مسیر Z به X بینهایت است. پس اینگونه Y دیگه از طریق X به Z نخواهد داشت چون بینهایت خواهد بود. فکر کنم اینطوری دیگه *loop back* نداشته باشیم.

این ایده، مشکل *count to infinity* را تا حدی حل میکند اما برای حالت هایی که بیشتر از دو روتر داریم، باز هنوز ممکن است مشکل داشته باشیم.

پس زمان همگرایی بردار فاصله با توجه به شرایط (خبر خوب یا خبر بد) متغیر خواهد بود.

تکنیک *split horizon*

این روش برای بهبود اون زمان ناپایداری سیستم آمده است.

به این شکل عمل میکند که اگر مسیری از یک همسایه گرفتد و کمینه مسیر شده است، آن مسیر را به همان همسایه، اطلاع نمیدهد. مثلاً اگر نود B مسیر $E, 2, A$ را در جدول دارد، میفهمد آن را از طریق A یاد گرفته است و بنابراین در پیامی که به A میدهد، مسیر $E, 2$ را درج نمیکند.

ترکیب این روش با *poison reverse* این تکنیک را قویتر میکند. مثلاً در مثال ما B مقدار E, ∞ را به A میدهد. نه تنها کمک نمیکند بلکه بی نهایتش می‌کند اون کمینه رو..

این دو روش فقط برای دو روتر جواب میدهند و نه بیشتر

سوال

روش *triggered update* بروز شدن با رخ دادن اتفاقات خاص و متدهای خاص را بررسی میکند. این متدها را بررسی کنید.

مقایسه دو روش LS و DV

- از دیدگاه پیچیدگی پیام:
در LS با n گره و E لینک $O(nE)$ پیام رد و بدل میشود.
در DS مبادله فقط بین همسایه هاست. و زمان همگرایی متغیر است.
- از نظر سرعت همگرایی:
در LS زمان اجرا الگوریتم از $O(n^2)$ است. برای انتقال همان nE پیام. البته از مشکل *oscillation* بین مسیرهای مختلف در اثر تعادل بار آنها، که صحبت کردیم، رنج میبرند.
زمان همگرایی DS متغیر است. ممکن است با مشکل *count to infinity* و *routing loops* مواجه شویم.
- از نظر استحکام در مقابل خرابی روترها:
 LS مقاوم تر است. چون در اینجا یک لینک نادرست آگهی میشود اما در DV ممکن است یک مسیر نادرست متشکل از چند لینک گزارش شود و اینگونه خطا در طول شبکه منتشر می‌شود.
در واقع در LS هر کس جدول خودش را محاسبه میکند ولی در DS هر نود برای محاسبه جدولش به اطلاعات جدول بقیه نودها نیاز دارد و اینگونه است که خطا *propagate* میکند به بقیه جدولها.

مراجع